Informatics II, Spring 2026, Exercise 1

Publication of exercise: February 23, 2026
Publication of solution: March 2, 2026
Exercise classes: March 2 - 6, 2026

Task 1

Consider the input array:

```
int array[] = {15, 5, 1, 4, 2, 8, 7};
```

Write a C program that implements the bubble sort algorithm to sort this array in ascending order. Your program should print the contents of the array after each full pass of the algorithm so that the sorting process can be observed.

Task 2

Consider the following sorting algorithm XSort with input array A:

```
Algo: XSort(A)
l = 1; r = n;
do
    for j = r to l + 1 do
        if A[j] < A[j-1] then
            exchange A[j] and A[j-1];
            m = j;
    l = m;
    for j = l to r − 1 do
        if A[j] > A[j+1] then
            exchange A[j] and A[j+1];
            m = j;
    r = m;
while l < r;
```

Given the input array $A = (4, 8, 8, 7, 7, 3, 5, 0, 5)$, show the state of array A when the the outermost loop is completed for the second time.

Task 3

Consider the Hilbert curves of order 1, 2 and 3 in Figure 1. Assume that each canvas on which the Hilbert curves are drawn is an identical square.

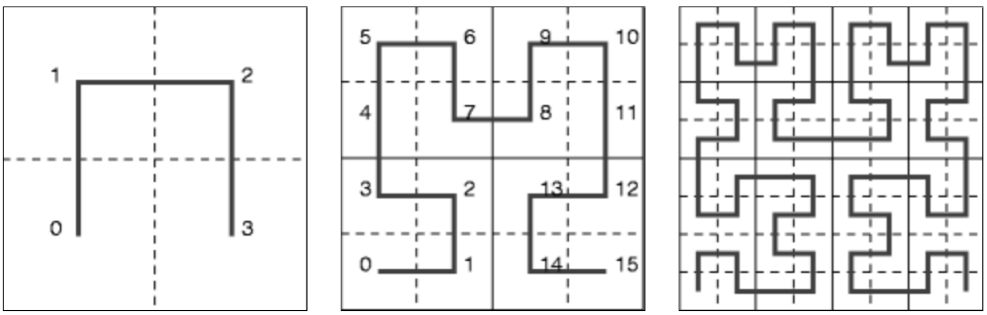

Figure 1: Hilbert curves of order 1, 2 and 3.

Determine the recursion scheme to compute the length of the Hilbert curve of order i on a square canvas with side length l. Then try to calculate the length of the Hilbert curve of order 15 on a $5cm \times 5cm$ square with either math or code.

Task 4

Write a C program that takes four integer numbers as input values on the command line. The four numbers are the values that shall be used to populate a 2×2 matrix. Compute the square of the matrix and print the original and the squared matrix side by side on the terminal.

The input handling is specific for exactly four values and does not have to be generalized. The matrix multiplication and the printing shall be general for $n \times n$ matrices.

Hint: Use `sscanf` to process command line arguments.

Task 5

Assume an array A with n integer elements and an integer c. The goal is to develop an algorithm that checks if there exists a pair of elements in A such that the sum of the two elements is equal to c. You can use the provided code skeleton `pairSum_skeleton.c`.

5.1 Develop and implement an algorithm `pairSum` that solves the problem in C.

5.2 Assume array A is sorted in non-decreasing order. Develop an algorithm `pairSumSorted` that exploits the sortedness and solves the problem faster than algorithm `pairSum`.

5.3 Assuming the input array A is sorted in non-decreasing order, determine the worst-case time complexity of both `pairSum` and `pairSumSorted`. In addition, quantify the difference in their worst-case wall-clock running times when the sorted array A contains 500,000 concrete numbers.